Localización y mapeo

Odometría · EKF · Landmarks · SLAM

Jhon Hesneydert Londoño · Juan David Villota

Robótica — Capítulo 6 · Corke, *Robotics, Vision and Control*

El problema: saber dónde está el robot

La odometría sola no alcanza: integra ruido. Los sensores ayudan, pero son parciales.

Solo odometría
Se integra el movimiento. El error crece y no hay forma de detectarlo.

Con landmarks
Las mediciones corrigen la pose, pero hace falta un mapa previo.

Sin mapa
Hay que estimar pose y mapa a la vez: SLAM.

Idea clave: cada parte añade una pieza —covarianza, corrección y mapa— y la pregunta es siempre la misma: ¿cuánto se sabe de la pose?

Parte 1 · Odometría: el error se acumula

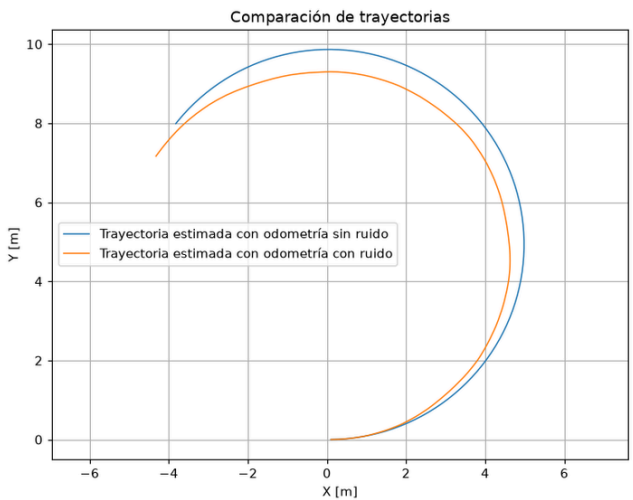
← Punto de partida de todas las partes: un robot que solo integra su propio movimiento.

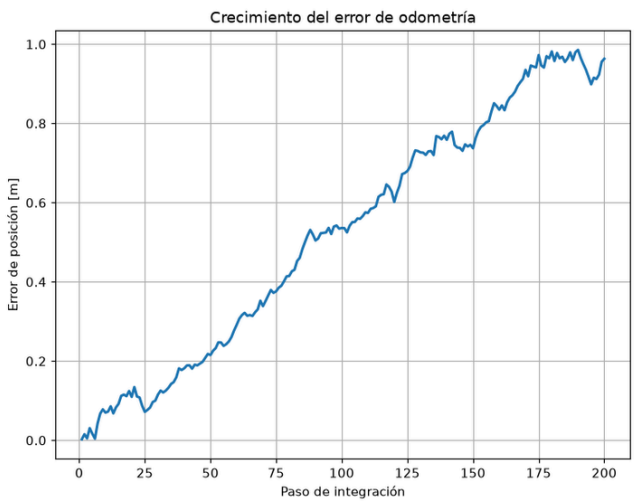
Qué hace el código

Dos robots avanzan 200 pasos con la misma entrada (1.0 m/s, 0.2 rad): uno recibe odometría con ruido y el otro perfecta. `robot.step()` genera la muestra y `robot.f()` integra la pose.

Por qué el error no se cancela

Cada muestra suma un error gaussiano independiente. Al integrar, los errores se acumulan: la pose derivada se aleja aunque el robot haga lo mismo.





¿Por qué la trayectoria presenta error? Cada paso suma ruido; la integración lo acumula y no se corrige nunca.

¿Qué ocurre si aumenta el ruido? La curva se separa antes y el error final crece.

¿Qué representa `V`? La covarianza del ruido de odometría: varianzas de velocidad y de timón.

Dato: 200 pasos; error final ≈ 1 m.

Parte 2 · EKF sin sensores: la incertidumbre crece

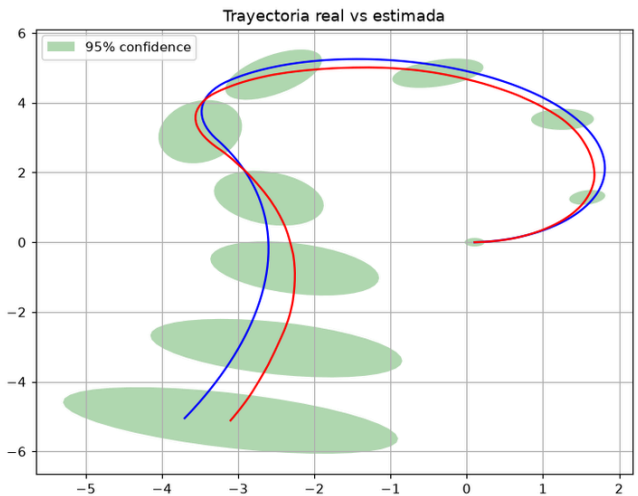
← Si la Parte 1 acumulaba error, aquí además se propaga la covarianza.

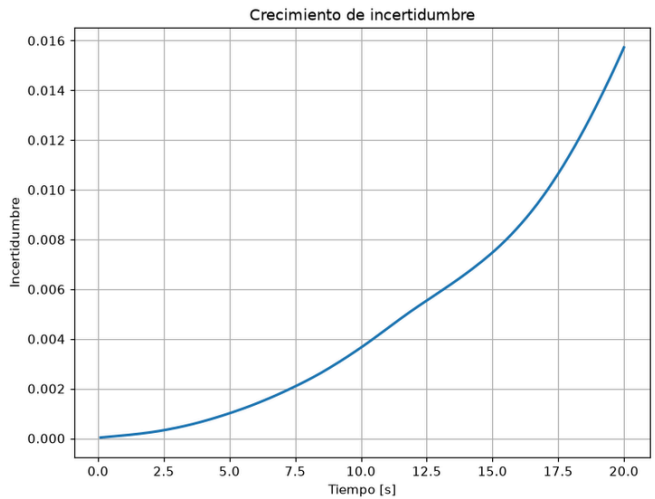
Qué hace el código

El filtro propaga la pose y su covarianza en cada paso (predicción). No hay sensores, así que nunca llega una corrección.

Por qué la curva solo sube

Sin observación no hay innovación (`v = 0`). A la covarianza se le suma un término positivo en cada paso: crece de forma monótona.





¿Por qué el EKF no corrige el error? Sin observaciones no hay innovación: el filtro solo propaga la odometría.

¿Qué ocurre si aumenta `V`? La covarianza crece más rápido; la curva se empina.

¿Qué representa la curva de incertidumbre? `sqrt(det P)` : crece de 3.3e-5 a 1.6e-2, unas 470 veces en 20 s.

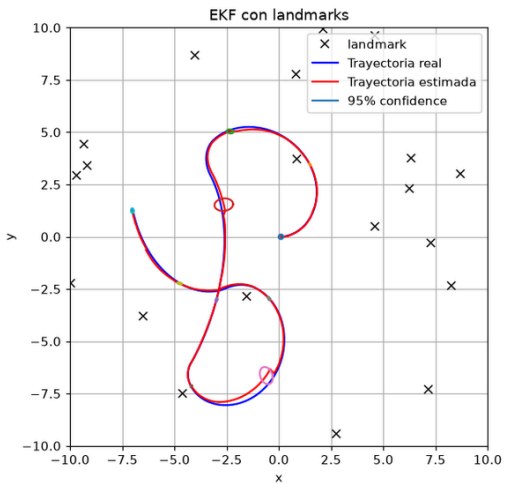
Dato: `T = 20 s` con `dt = 0.1` → 200 pasos.

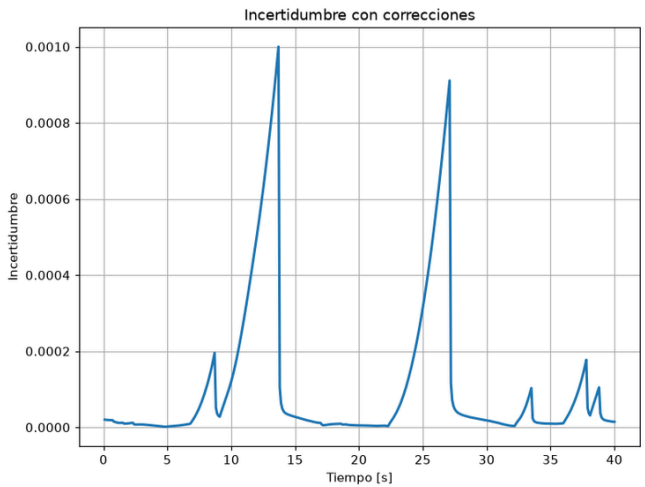
Parte 3 · Landmarks: la incertidumbre se acota

← Frente a la Parte 2, ahora hay mediciones externas que corrigen la pose.

Qué hace el código
Un sensor de rango y rumbo mide 20 landmarks conocidos. Con cada medición el filtro calcula la innovación y corrige con la ganancia $K = P H^T S^{-1}$.

Por qué baja la incertidumbre
Cada landmark aporta información sobre la pose y reduce la covarianza. Cuando el robot se aleja y deja de verlos, vuelve a crecer: de ahí los dientes de sierra.





¿Qué es la innovación? La diferencia entre lo medido y lo predicho: `z – h(x)`.

¿Qué ocurre cuando no se observan landmarks? Vuelve al caso de la Parte 2: la incertidumbre crece sin freno.

¿Por qué disminuye la incertidumbre? Cada landmark reduce la covarianza; los dientes de sierra son las correcciones.
Dato: 20 landmarks, alcance 4 m, apertura 180°.

Parte 4 · ¿Qué es SLAM?

← La Parte 3 usaba un mapa conocido; el SLAM quita esa referencia.

SLAM = estimar *a la vez* la pose del robot y el mapa, sin conocer ninguno de los dos de antemano.

Pose
Dónde está el robot en cada instante.

Mapa
Dónde están los landmarks que va viendo.

Asociación
Qué medición corresponde a qué landmark.

Qué cambia respecto a la Parte 3: `map=None`, el estado pasa de 3 a `3 + 2N` variables y no hay referencia externa que corrija la pose.

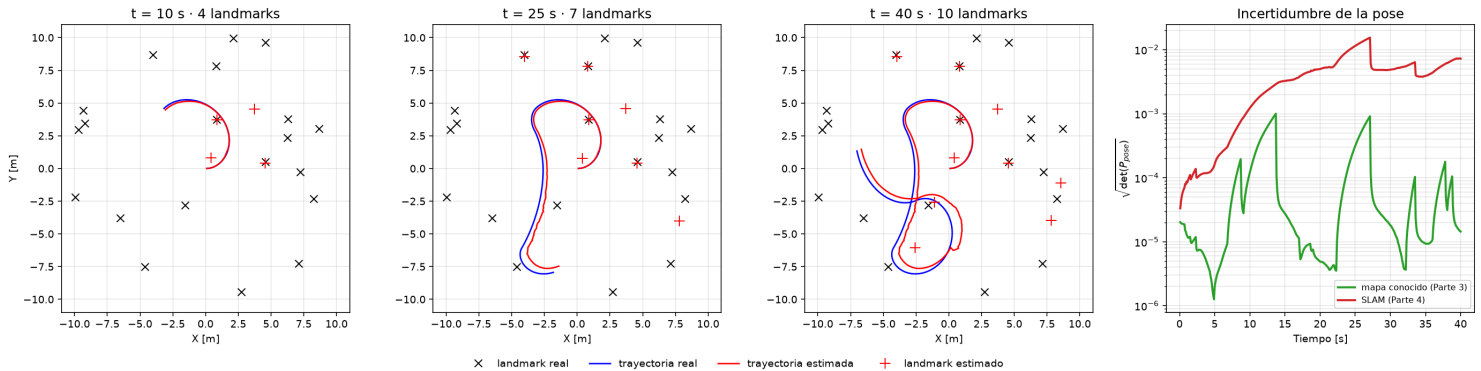
Lo que pide la guía para esta parte:

1. Construcción progresiva del mapa.
2. Corrección de la trayectoria.
3. Estimación simultánea de pose y landmarks.
4. Reducción parcial de la incertidumbre.
5. Diferencia entre trayectoria real y estimada.
6. Comparación con el EKF de la Parte 3.

La diapositiva siguiente muestra cada uno de esos seis puntos en la figura.

Parte 4 · Resultados del SLAM

← Cada resultado esperado de la guía se ve en un panel de la figura.



1 · Mapa progresivo
4 → 7 → 10 landmarks en t = 10, 25 y 40 s.

2 y 5 · Trayectoria
La roja sigue a la azul: se corrige al revisitar zonas y se ve la diferencia real/estimada.

3 · Pose y mapa
Las cruces rojas son los landmarks estimados; las negras, los reales.

4 y 6 · Incertidumbre
La curva roja (SLAM) queda por encima de la verde (P3): 7.3e-3 frente a 1.4e-5.

¿Por qué SLAM es más complejo que localización? Estado `3 + 2N` y asociación de datos.

¿Qué ventajas ofrece frente a un mapa conocido? No necesita mapa previo: se construye al recorrer.

¿Qué ocurre si aumenta el ruido del sensor? Se asocian peor los landmarks: el mapa se degrada y la pose deriva más.

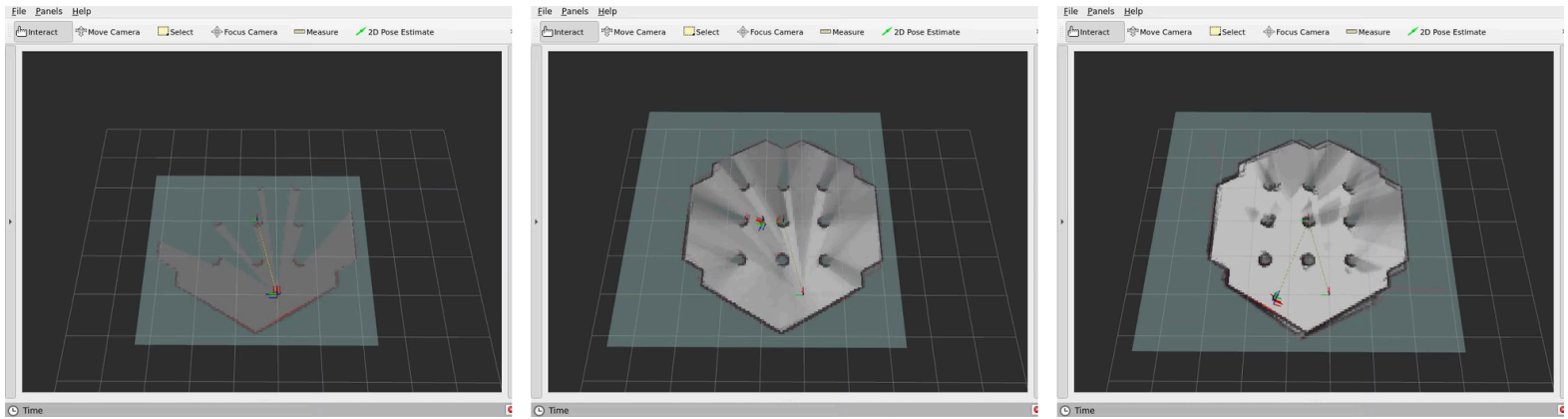
¿Qué aplicaciones industriales podrían usar SLAM? AGV/AMR en logística, limpieza, minería, agricultura, inspección.

¿Por qué la incertidumbre no desaparece del todo? Los landmarks también son incógnitas.

¿Cómo se compara con la Parte 3? 1.4e-5 con mapa conocido frente a 7.3e-3 en SLAM.

Cierre · la misma idea en ROS 2

← La Parte 4 estima pose y mapa en simulación; esta es esa misma idea corriendo sobre el robot.



Pipeline

LiDAR → SLAM → `/map` → RViz2. Gazebo entrega `/scan`, `/odom` y `/tf`; el robot se mueve con `/cmd_vel`.

RViz2

Mapa de ocupación, nube LiDAR, trayectoria y TF. El mapa se construye por zonas mientras el robot recorre el entorno.

Mapa

`map_saver_cli` exporta `my_map.pgm` y `my_map.yaml` a 0.05 m/celda, listo para reutilizar.

Grabación de la práctica de SLAM (ROS 2 Humble en Docker): SLAM 2D con Cartographer sobre TurtleBot3.

Limitaciones de lo presentado

Datos simulados
El ruido es gaussiano y los landmarks son puntos. No hay datos de un robot real ni errores sistemáticos.

Un solo escenario
La semilla es fija (0): las figuras son reproducibles, pero no son un estudio estadístico de muchas corridas.

Sensor idealizado
Rango de 4 m y apertura de 180°: el LiDAR del TurtleBot3 barre 360° y alcanza ~3.5 m.

Parte 5 no ejecutada
El pipeline en ROS 2 se muestra con la grabación de `Ref/`; no se presentó como práctica propia.

Dentro de esas condiciones, las cuatro partes reproducen los resultados que pide la guía y responden sus preguntas de análisis.

SLAM y navegación: cómo se complementan

La navegación decide el camino; el SLAM le da el mapa y la pose para poder decidirlo.

Cap. 5 · Navegación
Mapa dado y sin incertidumbre; se mide en pasos y optimalidad.

Cap. 6 · Localización y mapeo
Mapa construido y covarianza (`V` , P_0 , `W`); se mide el error de pose.

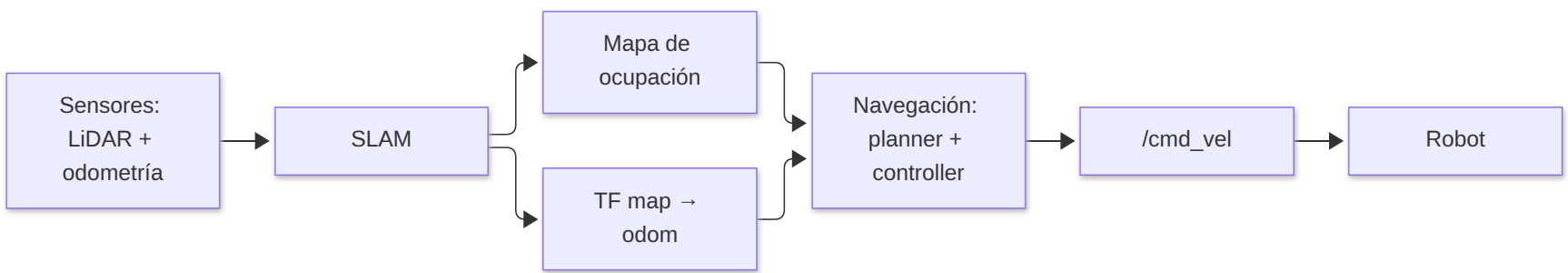
Juntos
El mapa del SLAM es la entrada que la navegación necesita.

Qué problema resuelve el SLAM

- **Localización:** ¿dónde estoy? → pose y TF `map → odom` .
- **Mapeo:** ¿cómo es el entorno? → mapa de ocupación.
- Funciona **sin mapa previo** y sin GPS: el robot entra a un lugar desconocido y lo cartografía.

Qué necesita la navegación

- **Un mapa** para planificar el camino global.
- **Una pose** para saber dónde está y cómo avanza.
- **Un costmap** actualizado para esquivar obstáculos.



Sin SLAM, el mapa y la pose se suponen dados (Capítulo 5). Con SLAM, se obtienen en línea y la navegación puede operar en entornos desconocidos.

Repositorio

github.com/juan-villota-b/Robotica-unal-informes

Guías, laboratorios, informes y presentaciones de Robótica.

Video · Mapeado con Cartographer: youtu.be/6sRefh7hCUA